

JSON & MongoDB

MongoDB



- One of the “NoSQL” poster children
- A “documents” database – documents are stored in collections of documents
- Started in 2007
- Targeting semi-structured data in JSON
- Designed to be easy to “scale out”
- Good support for indexing, partitioning, replication
- Nice integration in Web development stacks
- Not-so-great support for joins (or complex queries) or transactions (next topic!)

- *Document* - a way to organize and store data as a set of field-value pairs. {<field> : <value>, ...}
- *Field (keys)* - a unique identifier for a datapoint.
- *Value* - data related to a given identifier.
- *Collection* - an organized store of documents in MongoDB, usually with common fields between documents.
- There can be many collections per database and many documents per collection.

```
{“name” : “Alice”,  
  “title” : “team lead”}
```

```
{“name” : “Bob”,  
  “title” : “SW Engineer”}
```

Inside a MongoDB database

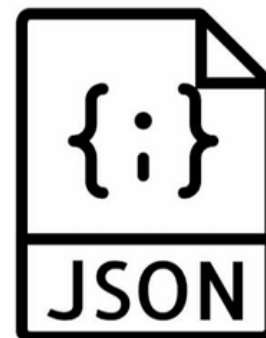
- **Database** = a number of “collections” ($\approx$ database)
- **Collection** = a list of “documents” ($\approx$ table)
- **Document** = a JSON object ($\approx$ row/tuple)
 - Must have an `_id` attribute whose value can uniquely identify a document within the collection

☞ In other words, a database has collections of similarly structured “documents”

- Much like tables of records, as opposed to one big XML document that contains all data
- Good reads
 - <https://www.mongodb.com/basics/json-schema-examples>

JSON (JavaScript Object Notation)

- Very lightweight data exchange format
 - Much less verbose and easier to parse than XML
 - Increasingly used for data exchange over Web: many Web APIs use JSON to return responses/results
- Based on JavaScript
 - Conforms to JavaScript object/array syntax—you can directly manipulate JSON representations in JavaScript
- But it has gained widespread support by all programming languages



MongoDB stores JSON data as BSON

- BSON = Binary JSON (binary encoding)
- JSON is text based (UTF-8 String encoding), space consuming, and limited data support (dates and binary not supported)
- BSON is optimized for speed, space, and flexibility
- JSON is Human and machine readable, BSON is machine readable
- MongoDB stores data in BSON internally and over the network
- JSON can be natively stored and retrieved in MongoDB
- <https://www.mongodb.com/json-and-bson>

Example JSON vs. XML

```
[
  { "ISBN": "ISBN-10",
    "price": 80.00,
    "title": "Foundations of Databases",
    "authors": [ "Abiteboul", "Hull", "Vianu" ],
    "publisher": "Addison Wesley",
    "year": 1995,
    "sections": [
      { "title": "Section 1",
        "sections": [
          { "title": "Section 1.1" },
          { "title": "Section 1.2" }
        ]
      },
      { "title": "Section 2" }
    ]
  }, ... ...
]
```

```
<bibliography>
  <book ISBN="ISBN-10" price="80.00">
    <title>Foundations of Databases</title>
    <author>Abiteboul</author>
    <author>Hull</author>
    <author>Vianu</author>
    <publisher>Addison Wesley</publisher>
    <year>1995</year>
    <section>
      <title>Section 1</title>
      <section><title>Section 1.1</title></section>
      <section><title>Section 1.2</title></section>
    </section>
    <section>
      <title>Section 2</title>
    </section>
  </book>
</bibliography>
```

JSON data model

```
[
  {
    "ISBN": "ISBN-10",
    "price": 80.00,
    "title": "Foundations of Databases",
    "authors": [ "Abiteboul", "Hull", "Vianu" ],
    "publisher": "Addison Wesley",
    "year": 1995,
    "sections": [
      { "title": "Section 1",
        "sections": [
          { "title": "Section 1.1" },
          { "title": "Section 1.2" }
        ]
      },
      { "title": "Section 2" }
    ]
  }, ...
]
```

(recursively defined)

- “Thing” = a **simple value** or an **array** or an **object**
- Simple types: numbers, strings (in double quotes), and special values “true”, “false”, and “null”
- Two basic constructs
 - **Array**: comma-separated list of “things” enclosed by brackets
 - Order is important
 - **Object**: comma-separated set of pairs enclosed by braces; each pair consists of an attribute name (string) and a value (any “thing”)
 - Order is unimportant
 - Attribute names “should” be unique within an object

Querying MongoDB

- `find()` and `sort()`
 - Analogous to single-table selection/projection/sort
- “Aggregation” pipeline
 - With “stages” analogous to relational operators
 - Join, group-by, restructuring, etc.

Try the queries yourself

```
[
{
  "ISBN": "ISBN-10",
  "price": 80.00,
  "title": "Foundations of Databases",
  "authors": [
    "Abiteboul",
    "Hull",
    "Vianu"
  ],
  "publisher": "Addison Wesley",
  "year": 1995,
  "sections": [
    {
      "title": "Section 1",
      "sections": [
        {
          "title": "Section 1.1"
        },
        {
          "title": "Section 1.2"
        }
      ]
    }
  ],
  {
    "title": "Section 2"
  }
]
},
{
  "ISBN": "ISBN-11",
  "price": 20.00,
  "title": "DBSTS",
  "authors": [
    "Ramakrishnan",
    "Gehrke"
  ],
  "publisher": "Addison Wesley",
  "year": 1999,
}
]
```

Basic MongoDB find()

- All books
`db.bib.find()`
- Books with title “Foundations of Databases”
`db.bib.find({ title: "Foundations of Databases" })`
- Books whose title contains “Database” or “database” and whose price is lower than \$50
`db.bib.find({ title: /[dD]atabase/, price: {$lt:50} })`
- Books with price between \$70 and \$100
`db.bib.find({ $and: [{ price: {$gte:70}}, { price: {$lte:100}}] })`
 - By the way, why wouldn't the following work?
`db.bib.find({ price: {$gte:70}, price: {$lte:100} })`
- Books authored by Hull
`db.bib.find({ authors: "Hull" })`
 - Note the implicit existential quantification
- Dot notation for specifying array elements
 - Books whose first author is Abiteboul
`db.bib.find({ "authors.0": "Abiteboul" })`
 - Note 0-based indexing; again, need to quote the expression

Note:

For our online Tester, “bib”
Has to be “collection”,
And regex does not work

find() with projection and sorting

- List just the book prices and nothing else
 - The (optional) second argument to find() specifies what to project: 1 means to return, 0 means to omit
 - `_id` is returned by default unless otherwise specified
- List books but not subsections, ordered by ISBN
- Output from find() is further sorted by sort() if you add `.sort({ISBN:1})`, where 1/-1 mean ascending/descending order
- ☞ “Aggregation pipelines” (next) are better suited for constructing more complex output

```
db.collection.find({  
  price: {  
    $exists: true  
  }  
},  
{  
  _id = 0,  
  price: 1  
})
```

```
db.collection.find({},  
{  
  "sections.sections": 0  
})
```

MongoDB aggregation pipeline

- Idea: think of a query as performing a sequence of “stages,” each transforming an input sequence of JSON objects to an output sequence of JSON objects
- “Aggregation” is a misnomer: there are all kinds of stages
 - Selection (`$match`), projection (`$project`), sorting (`$sort`)
 - Much of which `find()` and `sort()` already do
 - Computing/adding attributes with generalized projection (`$project/$addFields`), unnesting embedded arrays (`$unwind`), and restructuring output (`$replaceRoot`)
 - Operators to transform/filter arrays (`$map/$filter`)
 - Join (`$lookup`)
 - Grouping and aggregation (`$group`)
 - Operators to aggregate (e.g., `$sum`) or collect into an array (`$push`)

Grouping and aggregation

- Count books by publisher, and list the titles of books for each publisher

```
db.collection.aggregate([
  {
    $group: {
      _id: "$publisher",
      count: {
        $sum: 1
      },
      list: {
        $push: "$title"
      }
    }
  }
])
```

The required `_id` specifies the grouping expression, whose value becomes the identifying attribute of output objects (one per group)

Other attributes hold aggregate values, computed using aggregation operators

- *`$sum` compute a total by adding each input*
- *`$push` creates an array by appending each input*

Selection/projection/sorting

Find books published by “Addison Wesley”, output only their ISBN and title and price, sort by price

```
db.collection.aggregate([
  {
    $match: {
      "publisher": "Addison Wesley"
    }
  },
  {
    $project: {
      _id: false,
      ISBN: true,
      title: true,
      price: true
    }
  },
  {
    $sort: {
      price: -1
    }
  }
])
```

- *aggregate()* takes an array of stages
- Note quoting the dot notation
- Note the semantics of comparing a list of values: i.e., the query finds legislators who have ever served roles as PartyX